

Лабораторная работа №1

Вычисление числа π и подсчет простых чисел с использованием MPI

Автор: Фаст Никита

Дата: 2 мая 2026 г.

Содержание

1	Цель работы	2
2	Постановка задачи	2
3	Описание MPI-распараллеливания	2
3.1	Вычисление числа π	2
3.2	Подсчет простых чисел	2
4	Методика эксперимента	3
5	Результаты экспериментов	3
5.1	Вычисление числа π	3
5.2	Подсчет простых чисел	4
5.3	Ускорение на максимальных размерах задач	5
6	Анализ результатов	5
6.1	Вычисление числа π	5
6.2	Подсчет простых чисел	6
7	Вывод	7

1 Цель работы

Цель работы – изучить базовые средства стандарта MPI на примере двух вычислительных задач: численного вычисления числа π методом средних прямоугольников и подсчета количества простых чисел на интервале от 1 до n . В ходе работы необходимо реализовать MPI-распараллеливание, выполнить запуск программ при разном числе процессов и исследовать зависимость времени выполнения от размера задачи и числа процессов.

2 Постановка задачи

В лабораторной работе рассматриваются три практические задачи:

1. реализовать и запустить программу вычисления числа π по квадратурной формуле средних прямоугольников;
2. исследовать влияние параметра n и числа MPI-процессов на время работы программы вычисления π ;
3. реализовать параллельную MPI-программу для подсчета простых чисел на интервале $[1, n]$ и исследовать ее время выполнения при разных n и числе процессов.

3 Описание MPI-распараллеливания

3.1 Вычисление числа π

В программе `comp_pi.cpp` число π вычисляется по формуле

$$\pi \approx 4h \sum_{i=1}^n \frac{1}{1+x_i^2}, \quad x_i = h \left(i - \frac{1}{2} \right), \quad h = \frac{1}{n}. \quad (1)$$

Распараллеливание выполнено по слагаемым суммы. Каждый MPI-процесс получает свой ранг `myrank` и общее число процессов `nprocs`, после чего обрабатывает индексы

$$i = \text{myrank} + 1, \text{myrank} + 1 + \text{nprocs}, \text{myrank} + 1 + 2\text{nprocs}, \dots \quad (2)$$

Такое циклическое распределение дает почти одинаковое число итераций на процесс: различие между процессами не превышает одной итерации. Значение n вводится на процессе с рангом 0 или передается через аргумент командной строки, затем рассылается всем процессам с помощью `MPI_Bcast`. Каждый процесс вычисляет локальную частичную сумму, умножает ее на $4h$, после чего все частичные результаты суммируются на процессе 0 с помощью `MPI_Reduce` с операцией `MPI_SUM`.

Для измерения времени в программе используется `MPI_Wtime`. Перед началом измеряемого участка добавлен барьер `MPI_Barrier`, чтобы синхронизировать процессы и исключить из замера разброс времени старта отдельных процессов.

3.2 Подсчет простых чисел

В программе `count_primes.cpp` каждый процесс проверяет часть чисел из диапазона $[1, n]$. Распределение работы также выполнено циклически:

$$i = \text{rank} + 1, \text{rank} + 1 + \text{size}, \text{rank} + 1 + 2\text{size}, \dots \quad (3)$$

Для каждого числа вызывается функция `is_prime`. Она отбрасывает числа меньше 2, отдельно обрабатывает число 2, сразу исключает четные числа, а для нечетных чисел

проверяет делители от 3 до $\sqrt{i}$ с шагом 2. Локальное количество простых чисел хранится в `local_count`. После завершения проверки локальные счетчики суммируются на процессе 0 с помощью `MPI_Reduce`.

Схема обмена данными следующая:

- процесс 0 получает значение n из аргумента командной строки или со стандартного ввода;
- `MPI_Bcast` рассылает n всем процессам;
- каждый процесс независимо проверяет свою подпоследовательность чисел;
- `MPI_Reduce` суммирует локальные счетчики простых чисел.

В этой задаче вычислительная стоимость проверки разных чисел неодинакова. Четные числа отбрасываются почти мгновенно, а для простых и некоторых составных нечетных чисел требуется существенно больше проверок делителей. Поэтому равномерное распределение количества чисел не всегда означает равномерное распределение вычислительной нагрузки.

4 Методика эксперимента

Эксперименты проводились в следующей среде:

- операционная система: Windows 11;
- MPI-реализация: Microsoft MPI;
- компилятор: Visual Studio C++ compiler / MSVC;
- система сборки: CMake;
- запуск: `mpiexec`;
- число процессов: 1, 2, 4, 8.

Типовая команда запуска имела вид:

```
mpiexec -n <p> comp_pi.exe <n>
mpiexec -n <p> count_primes.exe <n>
```

Время выполнения измерялось средствами MPI через `MPI_Wtime`. Для задачи вычисления π измеряемый участок начинается после рассылки параметра n и синхронизации процессов. Для задачи подсчета простых чисел измеряется время от начала рассылки n до завершения редукции результата на процессе 0. Поэтому при малых n в результатах заметную долю занимает не собственно вычисление, а накладные расходы MPI: запуск процессов, обмены, синхронизация и редукция.

5 Результаты экспериментов

5.1 Вычисление числа π

В таблице 1 приведены времена выполнения программы `comp_pi.cpp` для разных значений n и числа процессов.

Таблица 1 – Время выполнения `comp_pi.cpp`, секунды

n	$p = 1$	$p = 2$	$p = 4$	$p = 8$
100	$2,01 \cdot 10^{-5}$	$9,9 \cdot 10^{-6}$	0,006 096 5	0,000 238 6
1000	$8,3 \cdot 10^{-6}$	$8 \cdot 10^{-6}$	$8,28 \cdot 10^{-5}$	0,000 267 8
10 000	$2,06 \cdot 10^{-5}$	$1,26 \cdot 10^{-5}$	$8,56 \cdot 10^{-5}$	0,000 231 3
100 000	0,000 102	$5,52 \cdot 10^{-5}$	0,000 124 8	0,000 194 9
1 000 000	0,000 933 7	0,000 472 9	0,000 347 5	0,000 481 3
10 000 000	0,009 394 5	0,004 639	0,004 714	0,002 653 3
100 000 000	0,092 105 6	0,048 076 5	0,043 363 2	0,024 541 5
1 000 000 000	0,923 468	0,475 725	0,334 414	0,244 949

На рисунке 1 показана зависимость времени выполнения от n для разного числа процессов.

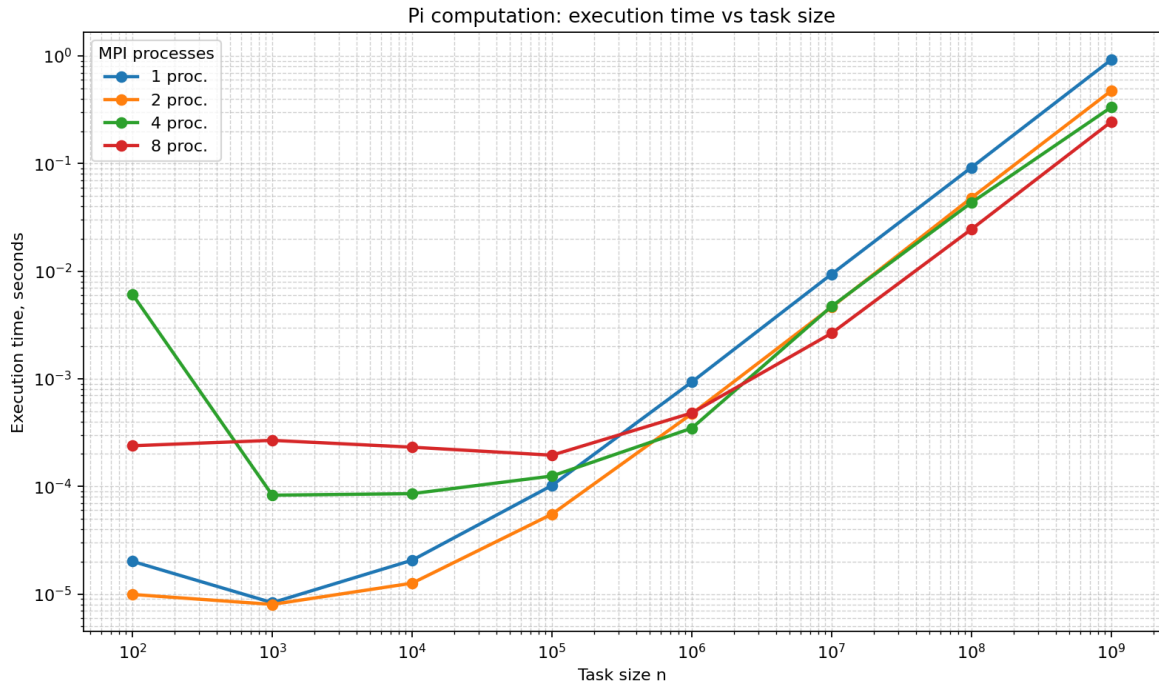


Рис. 1 – Время выполнения программы вычисления π

5.2 Подсчет простых чисел

В таблице 2 приведены времена выполнения программы `count_primes.cpp` для разных значений n и числа процессов.

Таблица 2 – Время выполнения `count_primes.cpp`, секунды

n	$p = 1$	$p = 2$	$p = 4$	$p = 8$
100	0,000 011	0,000 338	0,000 669	0,000 880
1000	0,000 018	0,000 294	0,000 514	0,000 888
10 000	0,000 117	0,000 321	0,000 557	0,000 895
100 000	0,002 108	0,002 207	0,001 427	0,001 415
1 000 000	0,048 270	0,047 102	0,024 503	0,016 213
10 000 000	1,190 101	1,166 540	0,676 874	0,403 796

На рисунке 2 показана зависимость времени выполнения от n для разного числа процессов.

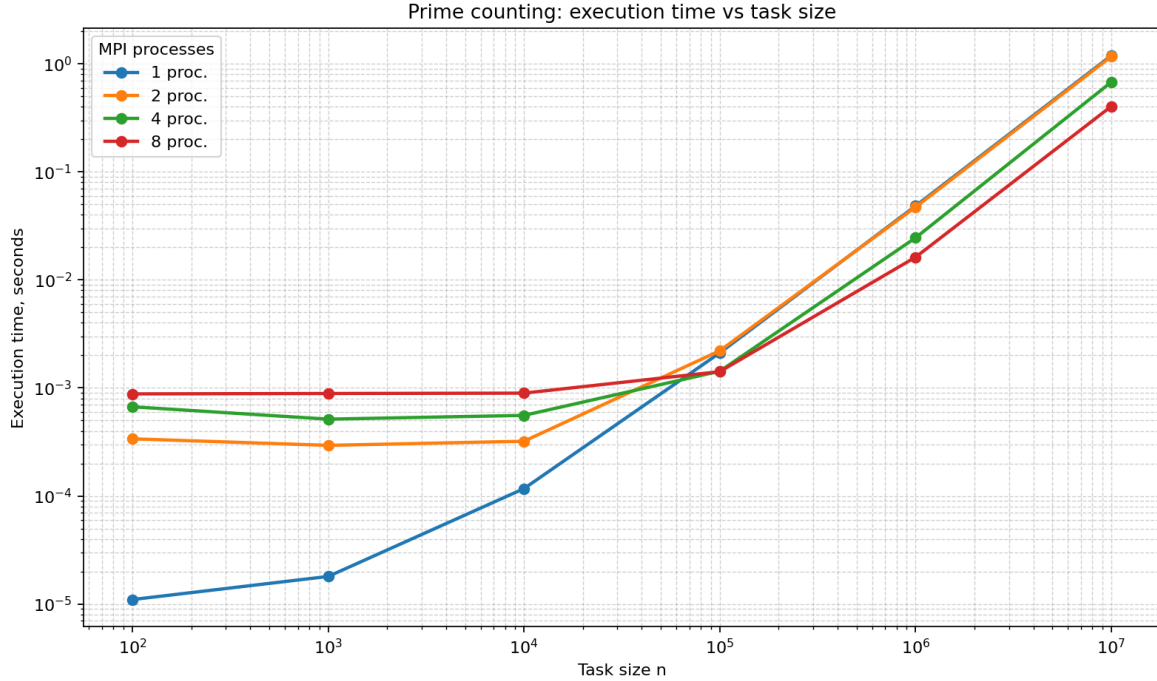


Рис. 2 – Время выполнения программы подсчета простых чисел

5.3 Ускорение на максимальных размерах задач

Для оценки масштабируемости рассчитано ускорение

$$S_p = \frac{T_1}{T_p}, \quad (4)$$

где T_1 – время выполнения на одном процессе, а T_p – время выполнения на p процессах.

Таблица 3 – Ускорение на максимальных значениях n

Задача	n	S_1	S_2	S_4	S_8
Вычисление π	1 000 000 000	1.00	1.94	2.76	3.77
Подсчет простых чисел	10 000 000	1.00	1.02	1.76	2.95

6 Анализ результатов

6.1 Вычисление числа π

Для малых значений n время выполнения находится на уровне микросекунд и миллисекунд. В этой области результаты нестабильны: накладные расходы MPI и погрешности измерения времени сравнимы с полезной вычислительной работой. Поэтому при $n = 100$, $n = 1000$ и $n = 10000$ увеличение числа процессов не всегда уменьшает время выполнения. Например, для $n = 100$ запуск на 4 процессах оказался значительно медленнее запуска на 1 и 2 процессах. Это не означает, что алгоритм не распараллеливается; это означает, что задача слишком мала для устойчивого измерения ускорения.

При росте n вычислительная часть начинает доминировать над накладными расходами. Для $n = 10^9$ время уменьшается с 0,923 468 с на одном процессе до 0,244 949 с на восьми процессах. Ускорение при этом составляет примерно 3,77. Оно меньше идеального ускорения 8, так как остаются последовательные участки, накладные расходы запуска и обменов MPI, а также ограничения аппаратной платформы: пропускная способность памяти, работа планировщика ОС и конкуренция процессов за вычислительные ресурсы.

В целом задача вычисления π хорошо подходит для MPI-распараллеливания, потому что каждая итерация имеет примерно одинаковую стоимость, а обмен данными минимален: один широковещательный обмен MPI_Bcast в начале и одна редукция MPI_Reduce в конце.

6.2 Подсчет простых чисел

Для задачи подсчета простых чисел при малых n параллельный запуск также невыгоден. Например, при $n = 100$ время на одном процессе равно 0,000 011 с, а на 8 процессах – 0,000 880 с. Полезная работа слишком мала, поэтому накладные расходы MPI полностью доминируют над вычислениями.

При больших n задача становится вычислительно тяжелее, однако масштабируемость остается хуже, чем в задаче вычисления π . Главная причина – неодинаковая стоимость проверки разных чисел на простоту. Четные числа отбрасываются одной проверкой $\text{num} \% 2 == 0$, тогда как нечетные числа требуют прохода по возможным делителям до $\sqrt{n}$ или до первого найденного делителя.

Особенно показателен случай двух процессов. При используемом циклическом распределении индексов процесс с рангом 0 получает числа

$$1, 3, 5, 7, \dots, \quad (5)$$

а процесс с рангом 1 получает числа

$$2, 4, 6, 8, \dots \quad (6)$$

Следовательно, при $p = 2$ один процесс обрабатывает почти все нечетные числа, то есть почти всю тяжелую часть работы, а второй процесс в основном быстро отбрасывает четные числа. Из-за этого при больших n время выполнения на 1 и 2 процессах почти совпадает. Например, при $n = 10^7$ время равно 1,190 101 с на одном процессе и 1,166 540 с на двух процессах, то есть ускорение составляет только около 1,02.

При 4 и 8 процессах часть нагрузки распределяется лучше, потому что среди процессов оказывается несколько рангов, получающих нечетные числа. Для $n = 10^7$ время уменьшается до 0,676 874 с при 4 процессах и до 0,403 796 с при 8 процессах. Однако ускорение остается неидеальным, потому что фактически основную работу выполняют только процессы, которым достаются нечетные числа. Кроме того, даже среди нечетных чисел стоимость проверки неодинакова: составные числа с малым делителем отбрасываются быстро, а простые числа требуют проверки всех возможных нечетных делителей до $\sqrt{n}$.

Для улучшения масштабируемости программы подсчета простых чисел можно изменить распределение работы. Например, можно заранее исключить четные числа и распределять между процессами только нечетные кандидаты. Другой вариант – использовать блочное распределение с учетом стоимости проверки или динамическое распределение заданий, при котором процессы получают новые диапазоны по мере завершения предыдущих. В рамках данной лабораторной работы реализован простой статический вариант, достаточный для демонстрации MPI-обмена и влияния балансировки нагрузки.

7 Вывод

В ходе лабораторной работы были реализованы и исследованы две MPI-программы: вычисление числа π методом средних прямоугольников и подсчет простых чисел на интервале от 1 до n . В обеих программах использованы базовые функции MPI: `MPI_Init`, `MPI_Comm_rank`, `MPI_Comm_size`, `MPI_Bcast`, `MPI_Reduce`, `MPI_Wtime` и `MPI_Finalize`.

Эксперименты показали, что для малых размеров задач параллельный запуск часто не дает выигрыша из-за накладных расходов MPI. Для больших n программа вычисления π демонстрирует устойчивое сокращение времени выполнения, так как работа хорошо балансируется между процессами и требует минимального обмена данными. В задаче подсчета простых чисел масштабируемость хуже: при двух процессах почти вся тяжелая работа приходится на процесс, обрабатывающий нечетные числа, поэтому времена для 1 и 2 процессов при больших n практически совпадают. Таким образом, эффективность MPI-программы определяется не только количеством процессов, но и качеством декомпозиции задачи, балансировкой нагрузки и отношением вычислительной работы к накладным расходам коммуникации.